ЛР 1. Твой код простыми словами

Памятка по варианту «Светофор»: какой файл открыть, что делает нужная функция и как всё связано.

Все пути ниже указаны относительно папки lab1. Номера слева в блоках кода - строки твоих исходников на 03.11.2026. Длинные строки перенесены для чтения.

Шесть основных частей

Файл	За что отвечает
Core/Src/main.c	Запускает программу и постоянно вызывает остальные части.
Core/Inc/board.h	Хранит номера выводов светодиодов и кнопки, а также время проверки нажатия.
Core/Inc/traffic.h Core/Src/traffic.c	Хранят длительности и правила: какой свет включён и когда его менять.
Core/Inc/button.h Core/Src/button.c	Проверяют нажатие. Не дают считать удержание кнопки за много нажатий.
Core/Inc/led.h Core/Src/led.c	Настраивают выводы светодиодов и передают команду включения в GPIO.
Core/Inc/gpio_driver.h Core/Src/gpio_driver.c	Твой драйвер выводов: настроить, прочитать и записать значения.

Как части вызывают друг друга

1. main.c читает кнопку через io_read_pin.
2. button_update решает, появилось ли новое нажатие.
3. traffic_update обновляет фазу светофора.
4. traffic_light сообщает, какой свет нужен сейчас.
5. led_write и io_write выставляют значения на выводах.

Почему рядом лежат .h и .c

В .h перечислены настройки, данные и названия доступных функций. В .c написано, что эти функции делают. Например, button.h объявляет button_update, а её действия находятся в button.c.

1. Где лежат настройки

board.h: к каким выводам обращаться

Core/Inc/board.h, строки 5-12

```
 5  #define BOARD_LED_PORT GPIOD
 6  #define BOARD_LED_MASK ((uint16_t)((1U << 13) | (1U << 14) | (1U << 15)))
 7  #define BOARD_GREEN ((uint16_t)(1U << 13))
 8  #define BOARD_RED ((uint16_t)(1U << 15))
 9  #define BOARD_YELLOW ((uint16_t)(1U << 14))
10  #define BOARD_BUTTON_PORT GPIOC
11  #define BOARD_BUTTON_PIN 15U
12  #define BOARD_DEBOUNCE_MS 30U
```

`BOARD_LED_PORT` - порт светодиодов. `BOARD_LED_MASK` выбирает сразу три нужных вывода. `BOARD_GREEN`, `BOARD_RED` и `BOARD_YELLOW` выбирают один из них. Запись `1U << 13` здесь означает «выбрать вывод 13».

По коду: зелёный - PD13, красный - PD15, жёлтый - PD14, кнопка - PC15. `BOARD_DEBOUNCE_MS = 30` задаёт, сколько миллисекунд новое положение кнопки должно сохраняться перед подтверждением.

traffic.h: сколько длится каждый этап

Core/Inc/traffic.h, строки 5-6

```
5  enum { GREEN_MS = 3000, RED_MS = 4 * GREEN_MS, SHORT_RED_MS = RED_MS / 4,
6         BLINK_MS = 3000, BLINK_HALF_MS = 500, YELLOW_MS = 1000 };
```

Имя	Значение в этой программе
`GREEN_MS`	3000 мс: обычный зелёный на 3 секунды.
`RED_MS` / `SHORT_RED_MS`	12 секунд / 3 секунды: обычный и сокращённый красный.
`BLINK_MS` / `BLINK_HALF_MS`	Мигание длится 3 секунды; свет меняется каждые 500 мс.
`YELLOW_MS`	Жёлтый на 1 секунду.

Если поменять `GREEN_MS`, вместе с ним изменятся `RED_MS` и `SHORT_RED_MS`: они вычисляются из него. Время мигания и жёлтого задано отдельно.

Здесь же перечислены фазы

Core/Inc/traffic.h, строки 7-9

`TRAFFIC_RED`, `TRAFFIC_GREEN`, `TRAFFIC_BLINK`, `TRAFFIC_YELLOW` идут именно в таком порядке. В объекте `Traffic`: `state` - текущая фаза, `since` - время её начала, `request` - сохранённая просьба сократить красный.

2. main.c: собирает всё вместе

Что выполняется один раз при запуске

Core/Src/main.c, строки 42-55

Строки	Что делает код
44-46	Подготавливает библиотеку HAL, счётчик времени и частоту работы. Настройка частоты находится в `SystemClock_Config`, строки 19-40.
47	`led_init` настраивает светодиоды. Начальное значение 0 выключает их до первого прохода цикла.
48-51	Включает порт кнопки и настраивает PC15 как вход. При ошибке вызывает `fatal(5)`.
52-55	Берёт текущее время, запоминает исходное положение кнопки и запускает светофор с красного.

Этот цикл повторяется всё время

Core/Src/main.c, строки 56-67

```
56      for (;;) {
57          now = HAL_GetTick();
58          bool press = button_update(&button, !io_read_pin(BOARD_BUTTON_PORT,
    BOARD_BUTTON_PIN), now);
59          traffic_update(&traffic, press, now);
60          uint16_t bits = 0;
61          switch (traffic_light(&traffic, now)) {
62          case LIGHT_RED: bits = BOARD_RED; break;
63          case LIGHT_GREEN: bits = BOARD_GREEN; break;
64          case LIGHT_YELLOW: bits = BOARD_YELLOW; break;
65          default: break;
66          }
67          led_write(&leds, bits);
```

57: получает время в миллисекундах. **58:** читает кнопку и получает `press`: новое нажатие есть или его нет. Знак `!` нужен потому, что нажатая кнопка на этой плате читается как 0.

59: передаёт нажатие светофору. **60-66:** превращает выбранный цвет в значения для выводов. При `LIGHT_OFF` остаётся `bits = 0`. **67:** отправляет эти значения светодиодам.

`&button`, `&traffic` и `&leds` означают: передать функции доступ к уже созданному объекту, чтобы она работала с его данными.

Ещё две полезные точки

`SysTick_Handler`, строка 71, вызывает `HAL_IncTick`: так обновляется время для `HAL_GetTick`. `fatal`, строки 13-17, сохраняет код ошибки в `lab1_fault` и останавливает дальнейшую работу. В обычном цикле ожидания через `HAL_Delay` нет.

3. button.c: одно нажатие - одно событие

`main.c` передаёт сюда уже понятный признак: `true` - кнопка нажата, `false` - отпущена. Модуль кнопки сам не читает PC15.

Какие данные запоминает кнопка

Core/Inc/button.h, строки 5-8

Поле	Простой смысл
`candidate`	Последнее положение, которое увидели на входе. Его ещё проверяем.
`changed_at`	Когда это положение появилось.
`pressed`	Положение, которое уже подтвердили.
`debounce_ms`	Сколько ждать подтверждения. В твоей программе 30 мс.

Сначала замечаем изменение

Core/Src/button.c, строки 7-12

```
 7  bool button_update(Button *b, bool raw, uint32_t now)
 8  {
 9      if (raw != b->candidate) {
10          b->candidate = raw;
11          b->changed_at = now;
12      }
```

Если вход `raw` отличается от сохранённого `candidate`, код запоминает новый вариант и время его появления. Каждое новое изменение начинает проверку заново.

Потом подтверждаем устойчивое положение

Core/Src/button.c, строки 13-17

```
13      if (b->candidate != b->pressed && (uint32_t)(now - b->changed_at) >=
    b->debounce_ms) {
14          b->pressed = b->candidate;
15          return b->pressed;
16      }
17      return false;
```

Если новое положение отличается от подтверждённого и держится уже 30 мс, код обновляет `pressed`. Запись `b->pressed` означает поле `pressed` у переданной кнопки.

При подтверждении нажатия возвращается `true`. При отпускании - `false`. При удержании положение уже не меняется, поэтому нового события нет: выполняется строка 17.

Что происходит сразу после включения

`button_init`, строки 2-6, записывает исходное положение в оба поля. Зажатая при запуске кнопка не даёт события: отпусти её и нажми снова.

4. traffic.c: выбирает фазу и цвет

`traffic_init`, строки 2-5, устанавливает красный, запоминает время старта и сбрасывает `request`. Дальше каждый проход главного цикла вызывает `traffic_update`.

Где принимается просьба сократить красный

Core/Src/traffic.c, строки 9-15

```
 9      if (press && t->state != TRAFFIC_GREEN) t->request = true;
10      uint32_t duration;
11      switch (t->state) {
12      case TRAFFIC_RED: duration = t->request ? SHORT_RED_MS : RED_MS; break;
13      case TRAFFIC_GREEN: duration = GREEN_MS; break;
14      case TRAFFIC_BLINK: duration = BLINK_MS; break;
15      default: duration = YELLOW_MS; break;
```

Строка 9: запоминает нажатие во всех фазах, кроме обычного зелёного. Мигание - отдельная фаза, поэтому в ней просьба принимается.

Строка 12: если `request` есть, красный длится `SHORT_RED_MS`, иначе `RED_MS`. Выражение с `?` и `:` здесь просто выбирает одно из двух времён. Остальные строки выбирают время своей фазы.

Где происходит переход дальше

Core/Src/traffic.c, строки 17-20

```
17      if ((uint32_t)(now - t->since) < duration) return;
18      if (t->state == TRAFFIC_RED) t->request = false; /* consumed by this red */
19      t->state = (TrafficState)(((unsigned)t->state + 1U) % 4U);
20      t->since = now; /* no shortened visible phases after a debugger pause */
```

17: если времени ещё мало, выйти из функции. **18:** уходя с красного, удалить уже выполненную просьбу. **19:** выбрать следующую фазу. `% 4U` возвращает счёт к нулю после четвёртой фазы, то есть после жёлтого снова будет красный. **20:** запомнить начало новой фазы.

Важная деталь кода: время красного считается от `since`, то есть от начала красного. Нажатие не запускает новые 3 секунды. Если красный идёт уже 8 секунд, подтверждённое нажатие переводит его в зелёный сразу.

Где сделано мигание

Core/Src/traffic.c, строки 28-29

```
28      case TRAFFIC_BLINK:
29          return ((uint32_t)(now - t->since) / BLINK_HALF_MS) % 2U ? LIGHT_OFF :
    LIGHT_GREEN;
```

В `traffic_light` эта строка чередует `LIGHT_GREEN` и `LIGHT_OFF` каждые 500 мс. В остальных фазах функция просто возвращает красный, зелёный или жёлтый: строки 25-27. Сама она фазы не переключает.

5. led.c и GPIO: команда доходит до ламп

led.c: короткая обёртка над драйвером выводов

Core/Src/led.c, строки 4-6

```
4      const IoConfig config = {IO_OUTPUT, IO_NO_PULL, 0, false, 0};
5      if (!led || !io_clock_enable(port)) return false;
6      if (io_configure(port, mask, &config, off_bits) != IO_OK) return false;
```

`led_init` выбирает режим выхода, включает порт и настраивает выводы. Затем сохраняет `port` и `mask` в LedBank: порт и набор выводов светодиодов (`Core/Inc/led.h`, строка 6).

Core/Src/led.c, строки 10-13

```
10  void led_write(const LedBank *led, uint16_t bits)
11  {
12      io_write(led->port, led->mask, bits);
13  }
```

`led_write` передаёт значения в `io_write`. Нужную комбинацию для цвета уже выбрал `main.c`; этот модуль работает с любыми индикаторами одного порта.

gpio_driver.c: где меняются значения выводов

Core/Src/gpio_driver.c, строки 21-24

```
21  void io_write(GPIO_TypeDef *p, uint16_t mask, uint16_t bits)
22  {
23      p->BSRR = ((uint32_t)(mask & (uint16_t)~bits) << 16) | (bits & mask);
24  }
```

`mask` выбирает выводы, которые разрешено менять. `bits` задаёт, какие из них получат 1; остальные выбранные получат 0. Запись в BSRR выполняет эти изменения за один раз.

Пример из твоей программы: `led_write(&leds, BOARD_RED)` выставит PD15 в 1, а PD13 и PD14 в 0. Другие выводы порта D эта запись не меняет.

Где читается кнопка

Core/Src/gpio_driver.c, строки 27-30

```
27  bool io_read_pin(GPIO_TypeDef *p, unsigned pin)
28  {
29      return pin < 16 && io_read(p, (uint16_t)(1U << pin)) != 0;
30  }
```

`io_read_pin` выбирает один вывод и возвращает `true`, если на нём 1. Вызванная внутри `io_read` (строка 25) берёт входные значения из IDR. Знак `!` в `main.c` превращает 0 от нажатой кнопки в `true`.

`io_read_latch` (строка 26) читает ODR: сохранённые выходные значения. Это проверка записанного значения, а не чтение кнопки.

6. gpio_driver.c: настройка выводов

Главная функция здесь - `io_configure`, строки 45-96. Она получает порт, выбранные выводы, настройки `IoConfig` и начальные выходные значения.

Откуда берутся настройки

Core/Src/main.c, строка 48

```
48      const IoConfig input = {IO_INPUT, IO_NO_PULL, 0, false, 0};
```

Поля идут в порядке из `Core/Inc/gpio_driver.h`, строки 8-14: режим, подтяжка, скорость выхода, `open_drain`, номер дополнительной функции. Эта строка выбирает обычный вход без внутренней подтяжки. Для кнопки используется внешний резистор на плате.

Что делает io_configure по порядку

Строки	Действие
47-49	Проверяет аргументы и блокировку настроек. При ошибке сразу возвращает результат, ничего не меняя.
53-73	Проходит по выбранным выводам. Читает старые настройки и готовит новые значения. Некоторые смены режима требуют отдельной подготовки.
74-86	Соблюдает порядок смены режима: например, отключает выход перед переводом во вход. Это служебная часть перенастройки.
88-95	Задаёт начальные значения выхода, записывает параметры и применяет режим. Затем возвращает `IO_OK`.

Core/Src/gpio_driver.c, строки 87-90

```
87      /* Preload latch BEFORE enabling output; update multiple signals atomically. */
88      if (c->mode == IO_OUTPUT) io_write(p, mask, initial);
89      p->OSPEEDR = (p->OSPEEDR & ~field_mask) | speeds;
90      p->PUPDR = (p->PUPDR & ~field_mask) | pulls;
```

Строка 88 загружает начальное значение **до включения режима выхода**. Сам режим записывается ниже, в `MODER`, строка 94. Вызов `io_configure` для светодиодов находится в `led.c`, строка 6.

Что означает результат функции

`IO_OK` - получилось. `IO_INVALID` - неверные параметры. `IO_LOCKED` - настройка выбранного вывода заблокирована. `IO_HANDOFF_REQUIRED` - перед этой сменой режима нужно отдельно подготовить вывод и связанное с ним устройство.

Остальные функции в этом файле

`port_index` (3-10) находит номер порта; `io_clock_enable` (12-19) включает его работу. `io_get_config` (35-44) читает текущие настройки. `io_write_pin` (31-34) меняет один вывод через общую `io_write`. Чтение и групповая запись разобраны на предыдущей странице.

7. Остальные файлы: для чего они нужны

Эти файлы помогают собрать, запустить и проверить работу. Основная логика твоего светофора находится в файлах с первой страницы.

Файл или папка	Простое объяснение
`Core/Startup/ startup_stm32f427xx.s`	Стартовый код: подготавливает запуск и вызывает `main`. Вызов `main` - строка 100.
`Core/Src/ system_stm32f4xx.c`	Стандартный системный файл ST. Помогает запустить контроллер и определить его текущую частоту.
`Core/Inc/ stm32f4xx_hal_conf.h`	Настройки подключаемых частей библиотеки HAL.
`Drivers/`	Готовые файлы CMSIS и HAL от производителя. В приложении они помогают с запуском и временем; работа GPIO написана отдельно в твоём драйвере.
`STM32F427VGTX_FLASH.ld`	Правила для сборщика: куда поместить программу и её данные в памяти контроллера.
`tools/build.py`	Собирает программу без окна IDE. С `--test` собирает отдельную программу тестов.
`tools/board.py SDK11M.cfg`	Скрипт проверки подключения, прошивки и запуска отладки. CFG описывает адаптер FT2232/JTAG.
`tests/test_lab1.c tests/run_tests.py tests/test.ld`	C-файл проверяет кнопку, светофор и GPIO. Python запускает тестовый ARM-код в эмуляторе. LD задаёт размещение тестовой программы.
`.project`, `.cproject`, `.settings/`, `lab1.launch`	Настройки проекта, сборки и запуска отладки в STM32CubeIDE.
`README.md`, `docs/ tools/render_schematic.py`	Описание работы, пояснения и схема стенда. Python-скрипт делает картинки страниц схемы; на светофор он не влияет.

Где посмотреть примеры проверок

В `tests/test_lab1.c`: `debounce_tests` начинается на строке 12, `traffic_tests` на 38, `gpio_tests` на 99. В `test_entry`, строки 154-159, эти три группы запускаются по очереди.

`tools/build.py` создаёт `build/lab1.elf`. Сборка CubeIDE создаёт `Debug/lab1.elf`. Это два разных пути; `board.py` по умолчанию берёт второй.

8. Что показать на защите

Один пример: нажали кнопку на красном

Пусть с начала красного прошла 1 секунда, и в этот момент программа уже подтвердила нажатие.

1. `main.c`**, строка 58.** `button_update` вернул `true`; в `press` появилось новое нажатие.

2. `traffic.c`**, строка 9.** `request` становится `true`: просьба сохранена.

3. `traffic.c`**, строка 12.** Вместо 12 секунд выбираются 3 секунды. `since` не меняется, поэтому остаётся ждать около 2 секунд.

4. `traffic.c`**, строки 17-20.** Когда пройдут 3 секунды от начала красного, фаза станет зелёной. Просьба сбросится, а `since` получит новое время.

5. `main.c`**, строки 61-67.** Выберется `BOARD_GREEN`. Через `led_write` и `io_write` включится нужный вывод.

Быстрая подсказка: где что найти

Нужно показать или изменить	Открой
Номера выводов и 30 мс проверки кнопки	`Core/Inc/board.h`, 5-12
Длительности и порядок фаз	`Core/Inc/traffic.h`, 5-9
Повторяющиеся действия программы	`Core/Src/main.c`, 56-67
Подтверждение нажатия и отсутствие повторов	`Core/Src/button.c`, 7-18
Сокращение красного и переход фаз	`Core/Src/traffic.c`, 9-20
Чередование света при мигании	`Core/Src/traffic.c`, 22-32
Передачу команды светодиодам	`Core/Src/led.c`, 10-13
Запись выводов / настройку выводов	`Core/Src/gpio_driver.c`, 21-24 / 45-96

Как коротко объяснить своё решение

«В `main` я связываю все части. Модуль кнопки сообщает только о новом подтверждённом нажатии. Модуль светофора хранит фазу и время её начала, выбирает следующий этап и нужный цвет. Модуль светодиодов передаёт команду моему драйверу GPIO, который записывает значения на выводы».

Памятка составлена по текущим исходникам и тестам в папке `lab1`. Все фрагменты взяты из файлов автоматически; исходный код при подготовке документа не менялся.